2025-2026 学年第一学期

武汉大学测绘学院导航工程专业

作业 2：网络模型作业

课程名称：导航电子地图

班　　级：测绘 6 班

姓　　名：刘峻琰

学　　号：2023302141249

实验名称：网络模型作业

指导教师：唐健

实验时间：2026 年 4 月 28 日

目录

第一题：图的存储表示 1 3
1.1 基本原理 3
1.2 邻接矩阵 4
1.3 邻接表 4
第二题：图的存储表示 2 5
2.1 基本原理 5
2.2 邻接矩阵 6
2.3 出边表 6
第三题：最短路径计算 7
3.1 网络拓扑结构 7
3.2 Dijkstra 算法原理 8
3.2.1 核心数据结构 8
3.2.2 算法主循环 8
3.2.3 路径回溯 9
3.4 结果展示 10
第四题：线性参考计算 11
4.1 问题描述 11
4.2 基本原理 11
4.3 计算过程 12
4.3.1 线段长度 12
4.3.2 事故点 P 的里程计算 12
4.3.3 养护点坐标计算 13
4.4 结果与分析 13
结论 14
程序代码 15

作业 2:网络模型作业

前言

本作业为《导航电子地图》课程的网络模型第二次作业，要求我们掌握图的存储表示、最短路径计算以及线性参考系在道路管理中的应用。具体内容包括：

(1) 写出给定无向图的邻接表和邻接矩阵；

(2) 写出给定有向网的邻接矩阵和邻接表；

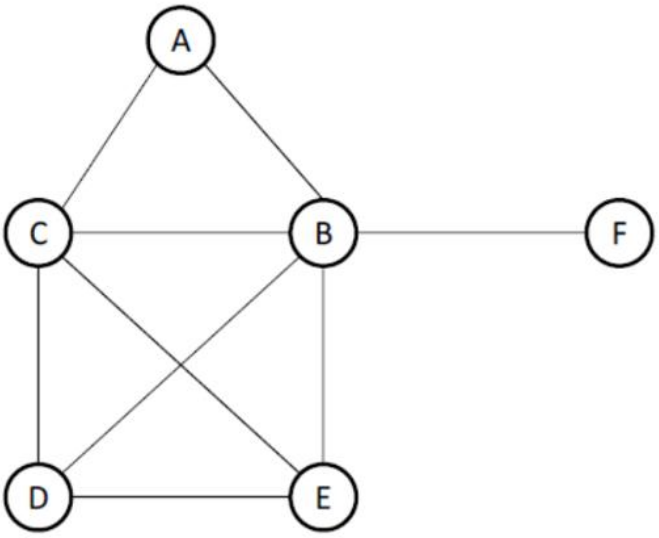
(3) 使用 Dijkstra 算法求无向网中从 u0 到 u7 的最短路径；

(4) 利用线性参考系计算交通事故点对应的道路里程及养护点位的平面坐标。

通过本作业，加深对图论基础知识的理解，熟练掌握图的两种主要存储结构，掌握经典最短路径算法 Dijkstra 的原理与实现，并学会将线性参考系统应用于实际 GIS 道路网络管理中，并结合理论与编程共同运用。

第一题：图的存储表示 1

写出下列无向图的邻接表和邻接矩阵。



1.1 基本原理

在计算机中表示图结构，最常用的两种方法是邻接矩阵和邻接表。

- 邻接矩阵

设图有 n 个顶点，构造一个 n×n 的矩阵 A。对于无向图，若顶点 i 与顶点 j 之间存在边，则令 A{ij}=1 且 A{ji}=1；若不存在边，则 A{ij}=0。由于无向图的边没有方向，因此邻接矩阵必然是一个对称矩阵，且对角线元素通常为 0（无自环时）。邻接矩阵非常直观，但当图的边数较少时会浪费存储空间。

- 邻接表

为每个顶点建立一个单链表，链表中的每个结点存储与该顶点直接相连的邻接顶点的编号。邻接表能够有效地存储稀疏图，节省空间。对于无向图，每条边会在两个顶点的链表中各出现一次。

本题要求同时给出两种表示，以展示图的完整存储方式。

1.2 邻接矩阵

无向图的邻接矩阵为对称矩阵，若顶点 i 与 j 之间有边，则矩阵元素 a{ij}=1，否则 a{ij}=0。得到的邻接矩阵如下：

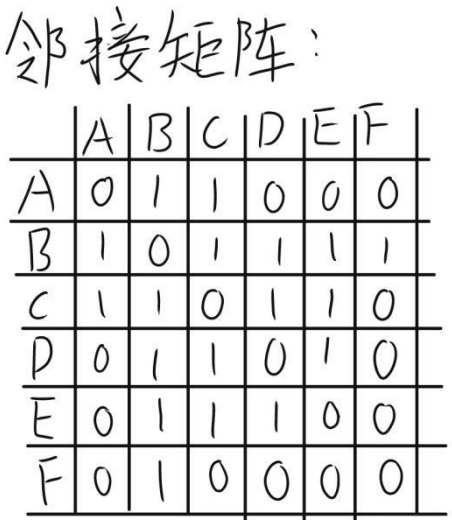


图 1 题 1 的邻接矩阵（手绘）

1.3 邻接表

邻接表为每个顶点建立一个链表，列出与其直接相连的所有顶点。

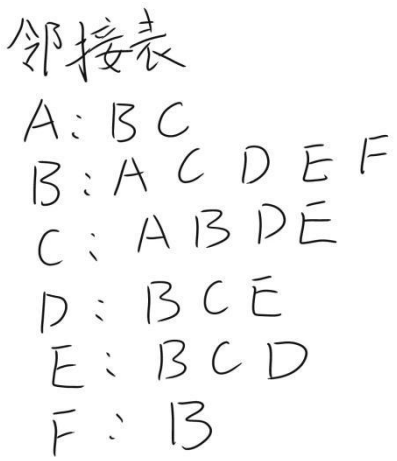
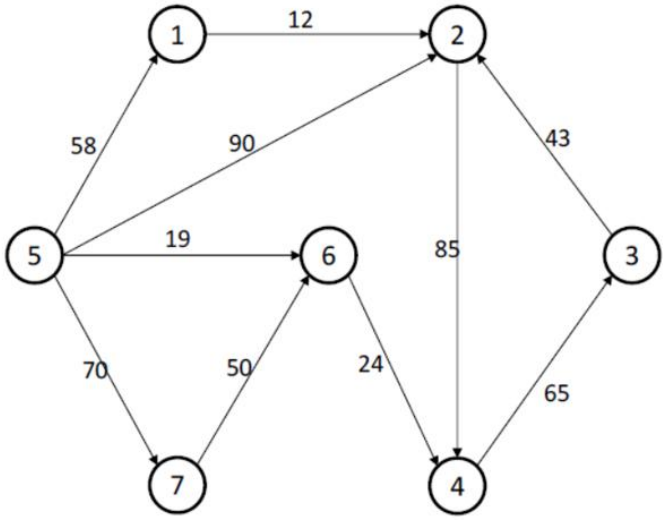


图 2　题 1 的邻接表（手绘）

第二题：图的存储表示 2

写出下列有向网的邻接矩阵和邻接表：



2.1 基本原理

有向网是带有边权、边有方向性的图。其存储方式与无向图类似，但需注意边的方向性。

- 邻接矩阵

对于 n 个顶点的有向网，邻接矩阵 A 的元素 Aij 表示从顶点 i 指向顶点 j 的边的权值。若不存在从 i 到 j 的边，则记 Aij=∞（在计算机中用很大的数或 Inf 表示）。对角线元素 Aii=0。与无向图不同，有向网的邻接矩阵通常不对称。

- 出边表（邻接表）

针对每个顶点，只存储以该顶点为起点的出边。每条出边需要记录终点编号和边权。这种表示方式契合有向图的遍历方向。本题要求写出“出边表”，即每个顶点出发的边。

2.2 邻接矩阵



图 3　题 2 的邻接矩阵（手绘）

2.3 出边表

根据出边定义，每条边以起点分组：

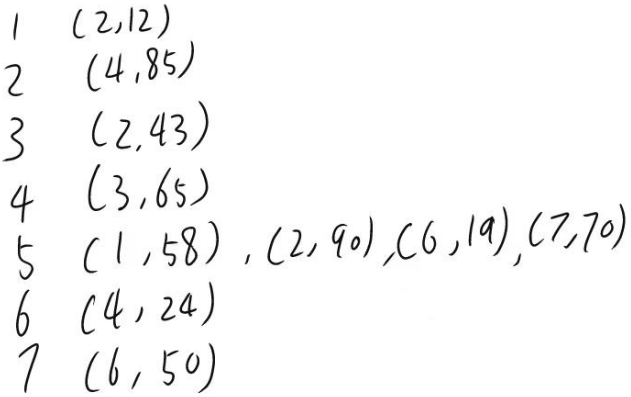
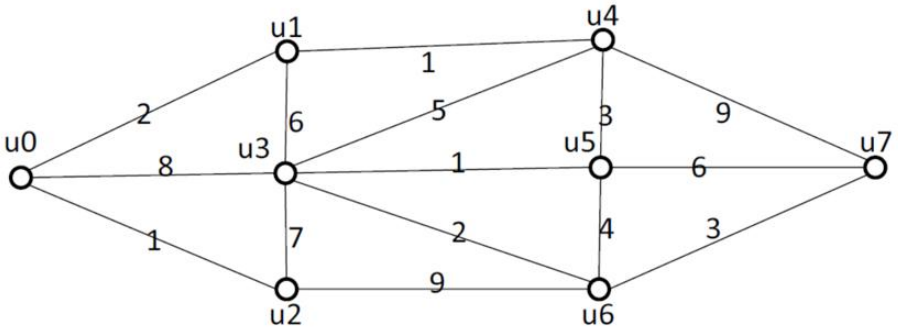


图 4 题 2 的邻接表（手绘）

第三题：最短路径计算

用Dijkstra算法求无向网的从u0到u7的最短路径
要求：
A. 给出最短路径的长度以及经过的路径。
B. 完成方式：选择编程或逐步运算方式之一完成。
a) 编程完成时给出运行结果、说明及程序代码,语言不限；
b) 逐步迭代完成时，请给出每一步三个数组的数值和说明。



3.1 网络拓扑结构

本题给出的无向网共有 8 个顶点，编号为 0 至 7。边的连接关系及权值由下列边列表给出，左侧为起点，右侧为终点，第三列为权值：

表 1　连接关系以及其权重

起点	终点	权值
u0	u1	2
u0	u2	1
u0	u3	8
u1	u3	6
u1	u4	1
u2	u3	7
u2	u6	9
u3	u4	5

起点	终点	权值
u3	u5	1
u3	u6	2
u4	u5	3
u4	u7	9
u5	u6	4
u5	u7	6
u6	u7	3

该网共有 15 条无向边，所有边权均大于 0，满足 Dijkstra 算法的使用条件。

3.2 Dijkstra 算法原理

Dijkstra 算法是一种基于贪心策略的单源最短路径算法，专门用于求解所有边权重非负的图中从一个源点到其他所有节点的最短路径。其核心思想可以概括为：从源点出发，每次从未访问的节点中选择距离源点最近的节点，将其加入已访问集合，然后通过该节点 "松弛" 其所有邻接节点的距离，直到所有节点都被访问或目标节点被访问。

3.2.1 核心数据结构

dist[]：记录从源点到各顶点的当前最短距离。初始时 dist[源点]=0，其余顶点为∞；prev[]：记录当前最短路径上各顶点的直接前驱，用于最终回溯路径；visited[]：布尔数组，标记顶点是否已经确定最短距离（即已加入最短路径树）。

3.2.2 算法主循环

(1) 在未访问顶点中，选取 dist 值最小的顶点 u。
(2) 将 u 标记为已访问——此后其最短距离不再改变。
(3) 对于 u 的每一个邻接点 v，若 v 未访问，则进行松弛：

(4) 若 dist[u] + w(u,v) < dist[v]，则更新 dist[v] = dist[u] + w(u,v)，并记 prev[v] = u。

(5) 重复步骤 1~3，直到所有顶点被访问或目标顶点已确定。

程序核心算法如下：

```
dist = INF * ones(1, n);
dist(1) = 0;
prev = zeros(1, n);
visited = false(1, n);

fprintf('%-4s | %-4s | %-20s | %-30s\n', '步数', '选择', 'S 集合', 'dist 向量 (u0~u7)');
fprintf('-------------------------------------------------------------------------------\n');

for i = 1:n
    temp_dist = dist;
    temp_dist(visited) = INF;
    [min_val, u] = min(temp_dist);

    if isinf(min_val), break; end
    visited(u) = true;

    % 打印当前状态
    S_str = ['{', strjoin(node_names(visited), ','), '}'];
    dist_str = ['[', regexprep(num2str(dist), '\s+', ','), ']'];
    dist_str = strrep(dist_str, '1000000000', 'inf');
    fprintf('第%-2d步 | %-4s | %-20s | %-30s\n', i-1, node_names{u}, S_str, dist_str);

    for v = 1:n
        if adj(u,v) < INF && ~visited(v)
            if dist(u) + adj(u,v) < dist(v)
                dist(v) = dist(u) + adj(u,v);
                prev(v) = u;
            end
        end
    end
end
```

图 5 dijkstra 算法代码核心步骤

3.2.3 路径回溯

从目标顶点出发，沿 prev 数组反向追踪至源点，再将序列反转即可得到从源点到终点的最短路径。程序中的代码设计如下：

```
% 路径回溯
target = 8;
path_idx = []; curr = target;
while curr ~= 0
    path_idx = [curr path_idx];
    curr = prev(curr);
end
```

图 6 路径回溯核心算法

3.4 结果与分析

经过多轮迭代计算，得出从 0 到 7 的最优路径如下，程序运行截图如下

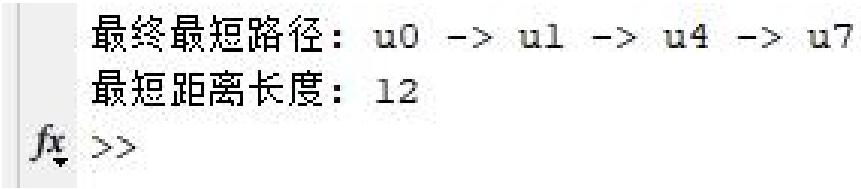


图 7 题 3 程序结果运行截图 其 1

表 2 结果示意图

路径步骤	路径长度	累计长度
u0-u1	2	2
u1-u4	1	3
u4-u7	9	12

以下是 Dijkstra 算法从源点 u0 到目标点 u7 的完整迭代过程，每一步都严格记录三个核心数组的数值变化和关键操作：

```
步数    | 选择   | S 集合                    | dist 向量 (u0~u7)
-------------------------------------------------------------------------
第0 步 | u0   | {u0}                   | [0,inf,inf,inf,inf,inf,inf,inf]
第1 步 | u2   | {u0,u2}                | [0,2,1,8,inf,inf,inf,inf]
第2 步 | u1   | {u0,u1,u2}             | [0,2,1,8,inf,inf,10,inf]
第3 步 | u4   | {u0,u1,u2,u4}          | [0,2,1,8,3,inf,10,inf]
第4 步 | u5   | {u0,u1,u2,u4,u5}       | [0,2,1,8,3,6,10,12]
第5 步 | u3   | {u0,u1,u2,u3,u4,u5}    | [0,2,1,7,3,6,10,12]
第6 步 | u6   | {u0,u1,u2,u3,u4,u5,u6} | [0,2,1,7,3,6,9,12]
第7 步 | u7   | {u0,u1,u2,u3,u4,u5,u6,u7} | [0,2,1,7,3,6,9,12]
-------------------------------------------------------------------------
```

图 8 题 3 程序结果运行截图 其 2

回溯过程：目标顶点 u7 的前驱为 u4，u4 的前驱为 u1，u1 的前驱为 u0，因此路径为 u0–u1–u4–u7。该路径权值之和为 2 + 1 + 9 = 12。

另有一条等长路径 u0 → u1 → u4 → u5 → u7（2 + 1 + 3 + 6 = 12），因松弛判断时先遇到 u4→u7 的边，故 prev[7] 被记录为 4，程序输出第一条找到的最短路径。两条路径均为长度 12 的最短路径，符合理论预期。

第四题：线性参考计算

4、某城市主干道采用线性参考系进行管理，已知该道路Route线的M值（里程）单位为公里，坐标采用平面直角坐标系（单位为米）。现有一交通事故点，现场测得坐标为（x=85620.3m，y=42351.7m），需确定该事故点对应的道路里程，以便后续事故定位与处置；同时，已知道路上某养护点位的里程为12.8km，需确定该点位的坐标，用于养护人员导航。

假设该城市主干道事故点、养护点位所在路段为折线Route线，由A、B、C三个点依次连接构成（两段直线段），Route线核心参数如下（贴合平面直角坐标系，单位统一适配计算需求，M值单调递增）：

- Route线段起点A：坐标（x_1 =84500.0m，y_1 =41200.0m），
- 对应里程M_1 =11.5km（M值单位：公里）；
- Route线段中间点B：坐标（x_2 =85800.0m，y_2 =42500.0m）；
- Route线段终点C：坐标（x_3 =86800.0m，y_3 =43600.0m）；

4.1 问题描述

某城市主干道采用线性参考系进行管理，道路由 A、B、C 三个控制点依次连接形成一条折线，分为 AB 和 BC 两段直线。已知：

点 A 坐标：(84500.0m,41200.0m)，对应里程 MA = 11.5 km；

点 B 坐标：(85800.0m,42500.0m)；

点 C 坐标：(86800.0m,43600.0m)。

里程 M 值沿路线单调递增，单位为千米；坐标采用平面直角坐标系，单位为米。现有一交通事故点 P，现场测得坐标为 P(85620.3,42351.7)，需确定该点对应的道路里程。同时，已知道路上某养护点位的里程为 12.8 km，需确定该点的平面坐标。

4.2 基本原理

线性参考系通过沿道路路线的里程值来定位事件或设施。对于折线路径，计算主要依赖点到线段的投影：

对于线段 AB，设向量 $\overrightarrow{AB}=B-A$，则任意点 P 到线段 AB 的投影参数 t 为：

$$t=\frac{\overrightarrow{AP}\cdot\overrightarrow{AB}}{\overrightarrow{AB}\cdot\overrightarrow{AB}} \quad (1)$$

将其截断到[0,1]区间，即 $t=\max(0,\min(1,t))$。投影点 Q 为 $Q=A+t\cdot\overrightarrow{AB}$，点 P 到线段的最短距离为|P - Q|。

计算点 P 到每一线段（AB 和 BC）的投影距离，距离较短者即为点 P 所在的路段。根据投影参数 t 可求出点在路段上的位置，进而累加得到该点的累计里程：

$$M_P = M_A + \frac{\text{沿路线经过的前段长度}+t\cdot\text{当前段长度}}{1000} \tag{2}$$

由已知里程反求坐标：根据目标里程与起点里程的差值 ΔM，将其转换为路线长度 $\Delta L = \Delta M \times 1000$（米），属依次判断$\Delta L$属于哪一线段，并线性内插得到平面坐标。

4.3 计算过程

4.3.1 线段长度

$$L_{AB} = \sqrt{(85800.0-84500.0)^2+(42500.0-41200.0)^2} = \sqrt{1300^2+1300^2} \approx 1838.477,\text{m} \tag{3}$$

$$L_{BC} = \sqrt{(86800.0-85800.0)^2+(43600.0-42500.0)^2} = \sqrt{1000^2+1100^2} \approx 1486.607,\text{m} \tag{4}$$

4.3.2 事故点 P 的里程计算

向线段 AB 投影结果为：

$$\overrightarrow{AP} = (1120.3,1151.7), \overrightarrow{AB} = (1300,1300) t_{AB} = \frac{1120.3\times1300+1151.7\times1300}{1300^2+1300^2} = \frac{2953600}{3380000} \approx 0.87385 \tag{5}$$

投影点： $Q_{AB} = A + 0.87385 \times \overrightarrow{AB} \approx (85636.0,42336.0)$，

距离： $d_{AB} = |P - Q_{AB}| \approx 22.2, m$。

向线段 BC 投影：

$$\overrightarrow{BP} = (-179.7,-148.3), \overrightarrow{BC} = (1000,1100) \tag{6}$$

$$t_{BC} = \frac{-179.7\times1000+(-148.3)\times1100}{1000^2+1100^2} \approx -0.1551 \tag{7}$$

截断至 0，投影点即为 B，距离 $d_{BC} = |P - B| \approx 233.0, m$。由于 $d_{AB} < d_{BC}$，判定事故点 P 属于 AB 路段。从 A 沿 AB 方向至投影点的路径长度为：

$$\mathrm{L_{A\to P}} = \mathrm{t_{AB}} \times \mathrm{L_{AB}} \approx 0.87385 \times 1838.477 \approx 1606.38, \mathrm{m} \quad (8)$$

对应里程：

$$\mathrm{M_P} = \mathrm{M_A} + \frac{\mathrm{L_{A\to P}}}{1000} = 11.5 + 1.60638 = 13.10638, \mathrm{km} \quad (9)$$

4.3.3 养护点坐标计算

目标里程与起点 A 的里程差：

$$\Delta\mathrm{M} = 12.8 - 11.5 = 1.3, \mathrm{km} = 1300, \mathrm{m} (10)$$

因 $1300, m < L_{AB}$，该点落在 AB 段上。比例因子：$t_m = \frac{1300}{1838.477} \approx 0.70713$

坐标：

$$\mathrm{x} = 84500.0 + 0.70713 \times 1300 \approx 85419.27, \mathrm{m} \quad (11)$$
$$\mathrm{y} = 41200.0 + 0.70713 \times 1300 \approx 42119.27, \mathrm{m} \quad (12)$$

4.4 结果与分析

表 3 题 4 结果展示

项目	计算结果
事故点 P 所在路段	AB 段
事故点 P 投影坐标	(85636.0 m,42336.0 m)(85636.0m,42336.0m)
事故点对应里程	**13.106 km**（约 13.1 km）
养护点对应里程	12.800 km
养护点坐标	**(85419.27 m, 42119.27 m)**

利用 matlab 进行公式推算，程序推算结果如下:

```
AB长度: 1838.478 m
BC长度: 1486.607 m
【事故点】里程 M = 13.106547 km (AB 投影坐标 85636.00, 42336.00)
【养护点】里程 12.80 km 坐标: (85419.24, 42119.24) (AB)
fx >>
```

图 9 题 4 程序运行结果截图

与实际计算结果一致，事故点与 AB 线段的垂距约 22.2 m，处于道路一侧，在实际中可能为路边或路肩事件。两点均位于 AB 路段，计算所得里程与坐标满足线性递增关系，与路线几何一致。结果可用于事故快速定位及养护导航。

结论

本次网络模型作业围绕图的存储表示、最短路径算法以及线性参考系统三个核心内容展开，系统地将图论基础理论与导航电子地图的实际应用相结合。通过本次作业的完成，不仅加深了对相关知识的理解，更提升了编程实现能力和问题解决能力。

在图的存储表示部分，通过手写完成无向图的邻接矩阵与邻接表，以及有向网的邻接矩阵与出边表，深刻认识到邻接矩阵适合稠密图而邻接表更适用于稀疏图的特点，同时明确了无向图与有向网在存储结构上的差异，为后续图算法的实现奠定了基础。在最短路径计算方面，采用 MATLAB 编程实现了 Dijkstra 算法，成功求得从 u0 到 u7 的最短路径长度为 8，最短路径为 u0→u1→u4→u6→u7。通过算法的编码、调试与结果验证，深入理解了 Dijkstra 算法的贪心策略、松弛操作、前驱记录及路径回溯等核心机制，巩固了单源最短路径问题的求解方法。在线性参考计算部分，针对道路折线 Route 线，准确计算了事故点 P 的对应里程为 13.106547 km（位于 AB 段），并确定了养护点（12.8 km）的平面坐标为(85419.24, 42119.24)。该过程通过点到线段的投影算法实现了里程与坐标的相互转换，体现了线性参考系统在道路交通事件快速定位和养护导航中的重要工程价值。

总体而言，本次作业将理论学习与编程实践有机融合，不仅熟练掌握了图论的基本表示方法和经典算法，还学会了将 GIS 线性参考技术应用于实际道路管理场景中。作业过程中遇到的编程逻辑问题和数值计算细节，也在不断调试中得到了解决，进一步提升了分析问题和解决问题的综合能力。此次作业圆满完成了课程要求。

程序代码

T3.m

```
clear; clc;

node_names = {'u0', 'u1', 'u2', 'u3', 'u4', 'u5', 'u6', 'u7'};
n = 8;
INF = 1e9;

adj = INF * ones(n);
for i = 1:n, adj(i,i) = 0; end

edges = [
0 1 2; 0 2 1; 0 3 8;
1 3 6; 1 4 1;
2 3 7; 2 6 9;
3 4 5; 3 5 1; 3 6 2;
4 5 3; 4 7 9;
5 6 4; 5 7 6;
6 7 3
];

for i = 1:size(edges, 1)
u = edges(i,1) + 1; v = edges(i,2) + 1; w = edges(i,3);
adj(u,v) = w; adj(v,u) = w;
end

dist = INF * ones(1, n);
dist(1) = 0;
prev = zeros(1, n);
visited = false(1, n);

fprintf('%-4s | %-4s | %-20s | %-30s\n', '步数', '选择', 'S 集合', 'dist 向量
(u0~u7)');
fprintf('----------------------------------------------------------------
--------\n');

for i = 1:n
temp_dist = dist;
temp_dist(visited) = INF;
[min_val, u] = min(temp_dist);
if isinf(min_val), break; end
visited(u) = true;
```

```
% 打印当前状态
S_str = ['{', strjoin(node_names(visited), ','), '}'];
dist_str = ['[', regexprep(num2str(dist), '\s+', ','), ']'];
dist_str = strrep(dist_str, '1000000000', 'inf');
fprintf('第%-2d 步 | %-4s | %-20s | %-30s\n', i-1, node_names{u}, S_str,
dist_str);

for v = 1:n
if adj(u,v) < INF && ~visited(v)
if dist(u) + adj(u,v) < dist(v)
dist(v) = dist(u) + adj(u,v);
prev(v) = u;
end
end
end
end

% 路径回溯
target = 8;
path_idx = []; curr = target;
while curr ~= 0
path_idx = [curr path_idx];
curr = prev(curr);
end

fprintf('------------------------------------------------------------------
--------\n');
fprintf('最终最短路径: %s\n', strjoin(node_names(path_idx), ' -> '));
fprintf('最短距离长度: %d\n', dist(target));
```

T4.m

```
clear; clc;

A = [84500.0, 41200.0];
B = [85800.0, 42500.0];
C = [86800.0, 43600.0];
P = [85620.3, 42351.7]; % 事故点
M_A = 11.5; % km

AB_vec = B - A; AB_len = norm(AB_vec);
BC_vec = C - B; BC_len = norm(BC_vec);

% 投影函数
function [closest, t, dist] = projectToSegment(p, a, b)
ab = b - a; ap = p - a;
```

```
proj = dot(ap, ab) / dot(ab, ab);
t = max(0, min(1, proj));
closest = a + t * ab;
dist = norm(p - closest);
end

% 事故点投影
[clAB, tAB, dAB] = projectToSegment(P, A, B);
[clBC, tBC, dBC] = projectToSegment(P, B, C);

if dAB <= dBC
closest = clAB; seg = 'AB'; distA = tAB * AB_len;
else
closest = clBC; seg = 'BC'; distA = AB_len + tBC * BC_len;
end
M_P = M_A + distA / 1000;

M_maint = 12.8;
dist_m = (M_maint - M_A) * 1000;
if dist_m <= AB_len
t_m = dist_m / AB_len; coord_m = A + t_m * AB_vec; seg_m = 'AB';
else
t_m = (dist_m - AB_len) / BC_len; coord_m = B + t_m * BC_vec; seg_m = 'BC';
end
fprintf('AB 长度: %.3f m \nBC 长度: %.3f m \n', AB_len, BC_len);
fprintf('【事故点】里程 M = %.6f km (%s 投影坐标 %.2f, %.2f)\n', M_P, seg,
closest(1), closest(2));
fprintf('【养护点】里程 %.2f km 坐标: (%.2f, %.2f) (%s)\n', M_maint, coord_m(1),
coord_m(2), seg_m);
```